

Warm up

Convert the following descriptions to ARMv8 assembly:

- x0 contains the address of an integer. Load the integer at that address into w0
- w0 contains an integer. Copy the integer in w0 to w1
- w0 contains an integer. Store the least significant byte of w0 at the address in x1

Warm up

- x0 contains the address of an integer. Load the integer at that address into w0
 - `ldr w0, [x0]`
- w0 contains an integer. Copy the integer in w0 to w1
 - `mov w1, w0`
- w0 contains an integer. Store the least significant byte of w0 at the address in x1
 - `strb w0, [x1]`

Precept 17: More ARMv8 and converting C to assembly

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

April 2, 2026

Overview

- 1 Walkthrough: assembly time, link time and run time
- 2 Control transfer
- 3 Summary

absval.s: assembly time

- The assembler's job is to translate `absval.s` into `absval.o` (object file)
- Object files have sections, denoted by the `.section` directive in the assembly code
 - `.section .text`: contains executable instructions
 - `.section .rodata`: contains read-only data (e.g., string literals)
 - `.section .data`: contains initialized data (e.g., global variables)
 - `.section .bss`: contains uninitialised data
- Heap and stack are managed at runtime, so do not have corresponding sections in the object file
- (The BSS is technically managed by the runtime loader, so the zeroed data is not physically stored in the object file, but the BSS section is still used to reserve space for it)
- See `absval.s` and `absval.c`

absval.s: interesting bits

- `promptStr::` defines a label for the string literal
- `.string "Enter an integer: ":` places string in rodata
 - Similarly for `scanfFormatStr` and `printfFormatStr`
- `iInput: .skip 4:` reserves 4 bytes of space to store `iInput` global variable
- `.equ MAIN_STACK_BYTECOUNT, 16:` defines a constant for the size of the main stack frame
- `.global main:` makes the main label visible to linker
- `.size main, (. - main):` defines the size of the main function as the distance from here (.) to the main label
 - Not generally necessary for execution, but needed for `gdb` and other debugging tools to know where the function ends=

absval.s: link time

- The linker creates an executable from the object files
- Resolution: resolves symbol references to their addresses
 - Find code referenced by global symbols, and merge into program image
 - Places definition of `_start()` into executable to serve as entry point
 - In this case:
 - ◊ Find `printf.o` (in `libc.a`) and merge its sections into `absval.o`
 - ◊ Similarly for `scanf` and `abs` functions
- Relocation: updates the code and data to reflect the resolved addresses
 - Replace reference to section offsets with real (virtual) memory addresses
 - Replace references to externally-defined functions with real (virtual) memory addresses
 - In this case:
 - ◊ In `b1`, replace reference to `printf` with a memory address
 - ◊ In `adr`, replace reference to `promptStr` with a memory address, etc.
 - ◊ Both are displacements from the program counter

absval.s: run time

- The executable is loaded into memory and executed by the OS
- Execution starts at the entry point defined by the linker (`_start()`)
 - `_start()` sets up the stack and stack pointer, and calls `main()`
- Function prolog: will discuss later

```
sub sp, sp, MAIN_STACK_BYTECOUNT
str x30, [sp]
```

- `adr x0, promptStr`: load address of `promptStr` into `x0`
- `bl printf`: branch and link to `printf`
 - `printf` uses contents of `x0` as argument (the string to print)
- Place arguments for `scanf` in `x0` and `x1`, then call `scanf`

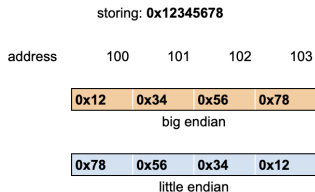
```
adr x0, scanfFormatStr
adr x1, iInput
bl scanf
```


absval.s: run time (cont.)

- `adr x0, iInput`: place the address of `iInput` in `x0`
- `ldr w0, [x0]`: load the four bytes at the address in `x0` (the user input) into `w0`
- `bl abs`: call `abs` function with the user input (in `w0`) as argument
 - Return value is placed in `w0`
- `adr x1, iAbsVal`: place the address of `iAbsVal` in `x1`
- `str w0, [x1]`: store the four bytes in `w0` (the result of `abs`) at the address held in `x1`
- ...
- `mov w0, 0`: place return value of `main` in `w0`
- Function epilog: reverse of prolog, will also discuss later

```
ldr x30, [sp]
add sp, sp, MAIN_STACK_BYTECOUNT
```
- `ret` returns from `main()` to `_start()`, which cleans up and exits

A note on endianness



- Endianness: the order in which bytes are stored in memory
 - Little endian: least significant byte stored at lowest address (little end)
 - Big endian: least significant byte stored at highest address (big end)
- ARMv8 is little endian by default
- In little endian, the least significant byte of a 4-byte integer is stored at the lowest address
 - Using `strb` and `ldrb` to store / load character data would work correctly, because the least significant byte of the integer is where the character data is stored
 - Not the case in big endian, since *most* significant byte of a 4-byte integer is stored at the lowest address

Walkthrough: `uppercase.s`, `rect.s`

- Walkthrough for `uppercase.s` and `rect.s`, pertinent points below
- `uppercase.s`: read character from `stdin`, convert to uppercase and print to `stdout`
 - `strb w0, [x1]`: store least significant byte of `w0` at address in `x1`
 - ◇ Should not use `str w0, [x1]` because that would store 4 bytes → data corruption or potential segfault
 - `ldrb w0, [x0]`: load the byte at the address in `x0` into the least significant byte of `w0`
 - ◇ Upper three bytes of `w0` zeroed out (bits 8-31 of `w0` zeroed by `ldrb`, bits 32-63 of `x0` zeroed by zero-extension when loading into `w0`)
 - ◇ Should not use `ldr w0, [x0]` because that would load 4 bytes → potential segfault
 - `mov w0, '\n'`: load ASCII value of newline character into `w0`
 - ◇ Can use C-style character literal in ARMv8 assembly
- `rect.s`: compute area of rectangle given width and height
 - `lsl w0, w0, 1`: logical left shift `w0` by 1 bit → multiply `w0` by 2
 - ◇ Bit-shifting is more efficient than multiplication
 - ◇ To multiply by powers of 2: use `lsl` (instead of `mul` or `add`)
 - ◇ To divide by powers of 2: use `lsr` or `asr`

Overview

- 1 Walkthrough: assembly time, link time and run time
- 2 Control transfer
- 3 Summary

Control transfer and flattening C code

- Control transfer instructions change the flow of execution
- Flattening C code involves converting structured control flow into a linear sequence of instructions
 - Eliminate nesting of control structures (e.g., if statements, loops) by using labels, `if` and `goto` statements
 - (Some consider labels and `goto` statements to be bad style)
- Why?
 - Nesting is difficult to translate to assembly code → flattening makes it easier to translate to assembly code
 - Each line of flattened C code corresponds to a small number of assembly instructions

Translating C to assembly

- Steps for translating C to assembly
 - Translate C code to flattened C code (using `if` and `goto`)
 - Build and run the flattened C code to verify correctness
 - Translate the flattened C code to assembly code
 - Correctness issues in assembly code are likely due to simple errors in translation
- General pattern: determine the condition under which you should *skip* the body of a control structure
 - `if (a > b){ body }` $\rightarrow$ run body if `!!(a > b)` $\rightarrow$ skip body if `!(a > b)`
 - But, this is not helpful for translating to assembly code, because we cannot directly translate `!(a > b)` to a single assembly instruction
 - Instead, we can rewrite the condition `!(a > b)` as `(a <= b)`
 - This translates to a single assembly instruction (e.g., `ble` for branch if less than or equal)
- See `powerflat.c` and `power.s`

Your turn: flattening C code

- Write flattened C code for the following if block:

```
if (a > b) {  
    temp = a;  
    a = b;  
    b = temp;  
}
```

- Write flattened C code for the following for loop:

```
for (i = 1; i <= exp; i++) {  
    power *= base;  
}
```

- Write flattened C code for the following while loop:

```
while (second != 0) {  
    temp = first % second;  
    first = second;  
    second = temp;  
}
```

Your turn: flattening C code (possible solutions)

```
if (a > b) {  
    temp = a;  
    a = b;  
    b = temp;  
}
```

```
for (i = 1; i <= exp; i++) {  
    power *= base;  
}
```

```
while (second != 0) {  
    temp = first % second;  
    first = second;  
    second = temp;  
}
```

```
1      if (a <= b) goto end;  
2      temp = a;  
3      a = b;  
4      b = temp;  
5  end:
```

```
1      i = 1;  
2  loop:  
3      if (i > exp) goto end;  
4      power *= base;  
5      i++;  
6      goto loop;  
7  end:
```

```
1  loop:  
2      if (second == 0) goto end;  
3      temp = first % second;  
4      first = second;  
5      second = temp;  
6      goto loop;  
7  end:
```


Overview

- 1 Walkthrough: assembly time, link time and run time
- 2 Control transfer
- 3 Summary

Summary

- How assembly code is assembled, linked and executed
- More examples of assembly code
- Converting C to assembly by flattening C code
- Dates to remember:
 - Assignment 4: due Tue, 4/7 at 9pm
 - Assignment 4 Quiz: Mon, 4/13 in lecture
- Acknowledgements
 - Most of the code in these slides is adapted from the precept materials, written by Prof. Bob Dondero and William Ughetta
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w10p2/>